VI. Bölüm - Grodient Descent ve Optimizasyon

Optimizasyon Problemi

* Optimizasyon, bir fonksiyonun mümkün olan en küçük veya en büyük değeri veren parametre-leri bulma sürecidir. Makine öğrenmesinde sistemi minimize etmek istenir.

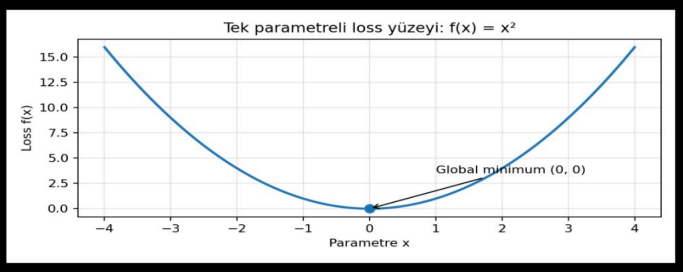
* Genel gösterim;

$$\min_{\theta} L(\theta)$$

Burda;
- θ = modelin tüm parametrelerini
- $L(\theta)$ = Tüm parametrelerden elde edilen loss

* Objective function optimize edilen genel ifadedir. Loss fuction modelin ne kadar yanlış yaptığını ölçer

Örnek: $f(x) = x^2$ fonksiyonunun minimumu $x=0$ $f(0)=0$
Bu sistemde negatif değer görülmez. Tek parametli losste minimu gbrnek kolay olur.



Şekil: Parabolün en alt noktası, loss'un minimumudur. Gerçek modellerde aranan şey yalnızca minimumun nerede olduğu değil, bilgisayarın o noktayı nasıl bulacağıdır.

Türev: Bulduğumuz noktadaki Eğim

* Türev, fonksiyonun seçilen noktadan hangi yöne ve nekadar hızla değiştiğini anlatır.

Türev	Fonksiyon x artarken	Loss'u azaltmak için
$f'(x) > 0$	Artar	x'i azalt (sola git)
$f'(x) < 0$	Azalır	x'i artır (sağa git)
$f'(x) = 0$	Anlık eğim sıfır	Durağan nokta; türünü incele

* İki nokta arasındaki kesen doğrun eğimi;

$$\frac{\Delta y}{\Delta x} = \frac{[f(x+\Delta x) - f(x)]}{\Delta x}$$

ile bulunur. Noktalar yaklaştıkça doğru kesen

Örnek: $f(x) = x^2$ türevini inceleyin

	f(x)	f'(x)	Yorum
-2	4	-6	Sağa
-1	1	-2	Sağa
0	0	0	Turunu incele
1	1	2	Sola git
2	24	6	Sola git

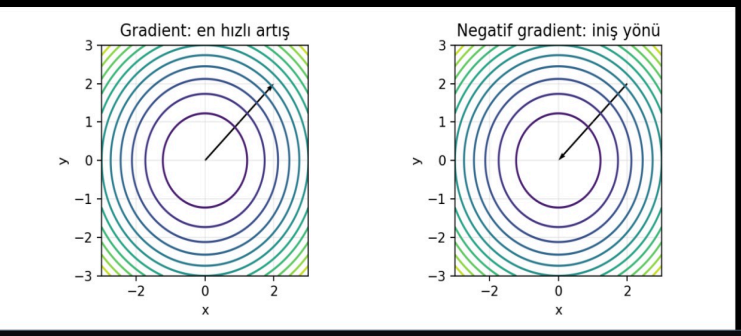
Gradient Vektörü

* Bütün kısmi türevleri bir araya getirilmiş halidir.

$$\nabla f = [\partial f/\partial x, \partial f/\partial y]^T$$

* Grodient bir sayı değildir. Her parametre eksenindeki eğimleri vektördür.

Örnek: $f(x) = x^2 + y^2$ için $\nabla f = (2x, 2y)$ ise $(3,4)$ noktasında $\nabla f(6,8)$ ve uzunluğu $\|\nabla f\| = \sqrt{6^2+8^2} = 10$ olur.



* Gradient, fonksiyonun en hızlı artığı yönü gösterir. Loss'u azaltmak istediğimiz için tam tersine, yani negatif grodient yönüyle devam edilir. Negatif Grandient en dik iniş yönüdür

Grodient Descent Algoritması

* Grodient descent, parametreleri loss'unu azaltacak yönde tekrar güncelleyen optimizasyon yöntemidir.

* Güncelleme kuralı:

$$\theta_{t+1} = \theta_t - \eta \cdot \nabla L(\theta_i)$$

Burda:

- θ_t = mevcut parametre
- η = öğrenme oranı (eta)
- ∇L_i = mevcut noktadaki göre grodient
- θ_{t+1} = bir sonraki parametre

Örnek: $f'(x) = 2x$ olsun başlangıç $=4$ $\eta = 0.1$ olduğuna göre 5 iterasyon ilerle

$$\theta_{t_1} = 4 - 0.1(2.4) = 3.2$$
$$\theta_{t_2} = 3.2 - 0.1(3.2 \cdot 2) = 2.56$$
$$\theta_{T3} = 2.56 - 0.1(2.5 \cdot 2) = 2.048$$
$$\theta_{T4} = 2.048 - 0.1(2.5) \cdot 2 = 1.6384$$
$$\theta_{T5} = 1.6384 - 0.1(1.6384 \cdot 2) = 1.31072$$

İterasyon	x	f(x)	f'(x)
0	4.00000	16.00000	8.00000
1	3.20000	10.24000	6.40000
2	2.56000	6.55360	5.12000
3	2.04800	4.19430	4.09600
4	1.63840	2.68435	3.27680
5	1.31072	1.71799	2.62144

İlk güncelleme: $x_1 = 4 - 0{,}1 \times 8 = 3{,}2$. Loss 16'dan 10,24'e iner. İkinci güncelleme: $x_2 = 3{,}2 - 0{,}1 \times 6{,}4 = 2{,}56$; loss 6,5536 olur. Her adımda minimuma yaklaşılır ve eğim küçüldükçe adımlar da küçülür.

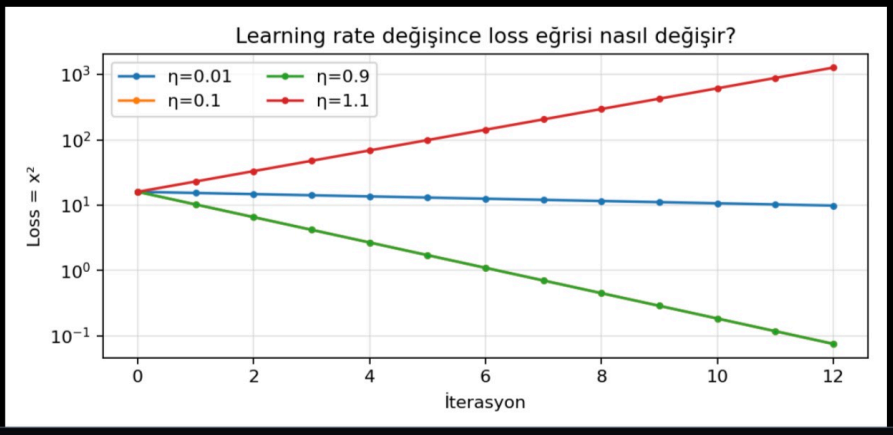
```
x = 4.0
eta = 0.1
for it in range(5):
    loss = x ** 2
    grad = 2 * x
    print(it, x, loss, grad)
    x = x - eta * grad
```

← kodu

Learning rate : Adım Büyüklüğü

*Gradient yönü söyler: Learning rate η ise o yönde nekor ileleyeceğini belirler. Aynı gradient ile farklı η değerleri çok farklı eğitim davranışı gösterir.

- Çok büyük η : Adımlar güvenilir çok kucuktur
- Uygun η : Loss kontrolü ve hızlı düşer
- Büyük η : Minimum üzerinden atlanır.
- Aşırı büyük η : Loss büyür , ıraksama olur.



$f(x)=x^2$ için $x_{t+1}=(1-2\eta)x_t$. Bu oyuncak problemde kararlılık koşulu $|1-2\eta|<1$, yani $0<\eta<1$'dir. $\eta=0{,}5$ seçilirse tek adımda $x=0$'a ulaşılır. Bu sınır yalnızca bu özel fonksiyon içindir; gerçek modellerde tek bir evrensel η sınırı yoktur.

η	Davranış	Yorum
0,01	Çok yavaş	Başlangıca yakın kalır
0,10	Düzgün yakınsama	Kontrollü iniş

η	Davranış	Yorum
0,90	Salınımlı yakınsama	Minimumun iki yanına geçer ama yaklaşır
1,10	Iraksama	Loss büyür

Salınım her zaman başarısızlık değildir: η=0,9 örneğinde parametre işaret değiştirerek salınır ama büyüklüğü küçülür. Iraksamada ise adımlar büyür ve loss artar.

Convergence ve Durma ölçütleri

*Convergence (yakınsama), parametrelerin ve lossun giderek durağanlaşması: yeni iterasyonların çok az değişiklik üretmesidir. Divergence (ıraksama), Lossun büyüyerek kontrolden çıkmasıdır.

Durum	Loss eğrisi	Öneri
Yakınsama	Düşer, sonra düzleşir	Eğitimi durdur / ölçütü kontrol et
Yavaş yakınsama	Yavaşça düşmeye devam eder	η'yi dikkatle artır veya iterasyon ekle
Salınım	Dalgalanır, genel eğilim aşağı olabilir	η'yi biraz küçült
Iraksama	Hızla büyür; inf/nan olabilir	η'yi küçült, gradient ve ölçekleri denetle

Yaygın durma ölçütleri: (1) maksimum iterasyon sayısına ulaşma, (2) gradient normunun $||\nabla L||<\varepsilon$ olması, (3) ardışık iterasyonlardaki loss farkının çok küçük olması, (4) validation loss'un belirli süre iyileşmemesi. Dördüncü durum early stopping olarak bilinir.

Gradient normu: $||\nabla L||=\sqrt{[(\partial L/\partial\theta_1)^2+(\partial L/\partial\theta_2)^2+\ldots]}$. Küçük norm, bulunduğumuz bölgenin düz olduğunu söyler; global minimum garantisi vermez.

Loss'un ilk iterasyondan itibaren nan olması veri/başlangıç hesaplarını düşündürür. Loss önce büyüyüp sonra nan oluyorsa learning rate'in aşırı büyük olması sık görülen sebeptir. Ayrıca işaret hatası, feature ölçekleri ve uç değerler kontrol edilmelidir.

```
loss = np.mean(error ** 2)
if not np.isfinite(loss):
    print("Loss sonlu de■il; learning rate ve gradient hesab■n■ kontrol et.")
    break
```

iteration ve epoch

* iteration, bir parmetre güncelemesidir. Epoch, eğitim verisinin tamamını birkez baştan sona işlenmesidir. Bir epoch içindeki iterasyon sayısı batch size bağlıdır.

$$\frac{\text{iterasyon}}{\text{epoch}} = \text{ceil}\left(\frac{\text{örnek sayısı}}{\text{batch size}}\right)$$

Son batch eksik kalabileceğinden yuvarlanır.

Örnek sayısı	Batch size	1 epoch	10 epoch
1.000	100	10 iterasyon	100 iterasyon
1.000	32	32 iterasyon	320 iterasyon
1.000	1	1.000 iterasyon	10.000 iterasyon
50.000	64	782 iterasyon	7.820 iterasyon

Örneğin 800 örnek ve batch size=200 için bir epoch 800/200=4 iterasyondur. 20 epoch toplam 80 parametre güncellemesi demektir. Batch size=1 olsaydı bir epoch 800 iterasyon olurdu.

Batch, Stochastic ve mini-batch gradient descent

Yöntem	Gradient kaç örnek	Güncelleme sıklığı	Özellik
Batch GD	Tüm eğitim verisi	Epoch başı 1	Kararlı, daha fazla bellek
Stochastic GD	Tek örnek	Her örnekten sonra	Gürültülü sık güncelleme
Mini batch	Küçük grup	Her grup sonrası	Hız ve kararlılık dengesi

* Mini batch makine öğrenmesinde yaygındır. SGD'nin gürültülü güncelleme bazen sığ yerel minumunlardan çıkmaya yardımcı olabilir. mini-batch'se daha dengeli yön tahmini yapar.

Loss yüzeyin şekilleri

* Convex (dış bükey), Loss yüzeyi çanak biçimindedir; uygun koşullarda global minimu vardır. Örneğin; Doğrusal Regresyon ve MSE örnektir.

* Non-convex yüzeyde birden fazla çukur ve tepe olabilir; başlangıç noktası sonucu değişebilir. Sinir ağları bu tür yüzeylere örnektir.

Özellik	Convex	Non-convex
Minimum	Tek global minimum (tipik çanak örneği)	Birden fazla yerel/global minimum olabilir
Başlangıç noktası	Uygun koşullarda sonucu değiştirmez	Sonucu etkileyebilir
Garanti	Uygun η ile global minimuma yakınsama	Genel global minimum garantisi yok
Örnek	Linear regression + MSE	Sinir ağları

- Global minimum: Tüm tanım kümesindeki en düşük loss.
- Local minimum: Yakın çevresinde en düşük noktadır; uzakta daha düşük bir nokta bulunabilir.
- Saddle point (eyer noktası): Bir yönde minimum, başka bir yönde maksimum gibi davranır. Gradient sıfır olabilir ama nokta minimum değildir.

Örnek eyer fonksiyonu $f(x,y)=x^2-y^2$. Orijinde gradient sıfırdır. x ekseni boyunca çukur, y ekseni boyunca tepe davranışı gösterir. Bu nedenle "gradient sıfır = iş bitti" denemez.

Yüksek boyutlu problemlerde eyer noktalarının çevresindeki geniş ve düz bölgeler de öğrenmeyi yavaşlatabilir; gradient çok küçüldüğü için ilerleme durmuş gibi görünür.

Lineer Regresyon'un gradient descent ile eğitme

*Tek değişkenli doğrusal model: $y_i = \beta_0 + \beta_1 \cdot x$. MSE loss $L(\beta_0,\beta_1) = \frac{1}{n}\Sigma(y-y_i)^2$

Burda hata konvalizasyonu $e = \hat{y}_i - y_i$ olarak alınır.

Kısmi türevler:

$$\partial L/\partial \beta_0 = (2/n)\Sigma(\hat{y}_i - y_i)$$

$$\partial L/\partial \beta_1 = (2/n)\Sigma(\hat{y}_i - y_i)x_i$$

β_0 gradient'i hataların ortalamasının iki katıdır. β_1 gradient'inde hatalar x_i ile ağırlıklandırılır; x'i büyük örnekler eğim parametresinin güncellenmesinde daha fazla etkiye sahip olabilir.

Çok önemli uygulama ayrıntısı: İki gradient de eski β_0 ve β_1 değerleri kullanılarak hesaplanır. Önce iki gradient değişkene atanır, ardından iki parametre birlikte güncellenir. β_0'ı güncelleyip yeni β_0 ile β_1 gradient'ini hesaplamak aynı güncelleme adımı değildir.

Örnek: $x = [1,2,3]$ $y = [2,4,6]$ başlangıç $\beta_0 = 0$ $\beta_1 = 0$; $\eta = 0.1$

x	y	$y = \beta_0 - \beta_1 \cdot x$	$e = \hat{y}_i - y_i$
1	2	0	-2
2	4	0	-4
3	6	0	-6

Başlangıç MSE: $\frac{(2)^2 + 4^2 + 6^2}{3} \cong 16,6667$

β_0 gradient $= 2 \overset{mean}{\cdot} (-2)\ (-4)\ .(-6) = (-4).2 = -8$

β_1 gradient $= 2\ mean\ (-2 \times 1 ; -4 \times 2 ; -6 \times 3) = 2 \times (-\frac{28}{3}) = 18,6667$

Numpy uygulama

Ders dokümanındaki mini lab verisi: x=[1,2,3,4,5,6,7,8], y=[3,5.18,6.84,8.47,10.73,12.41,15.04,17.8]. Başlangıç $\beta_0=\beta_1=0$, $\eta=0,01$ ve 1000 iterasyon kullanılır.

```
import numpy as np
import matplotlib.pyplot as plt

x = np.array([1., 2., 3., 4., 5., 6., 7., 8.])
y = np.array([3., 5.18, 6.84, 8.47, 10.73, 12.41, 15.04, 17.8])

b0, b1 = 0.0, 0.0
lr = 0.01
loss_history = []

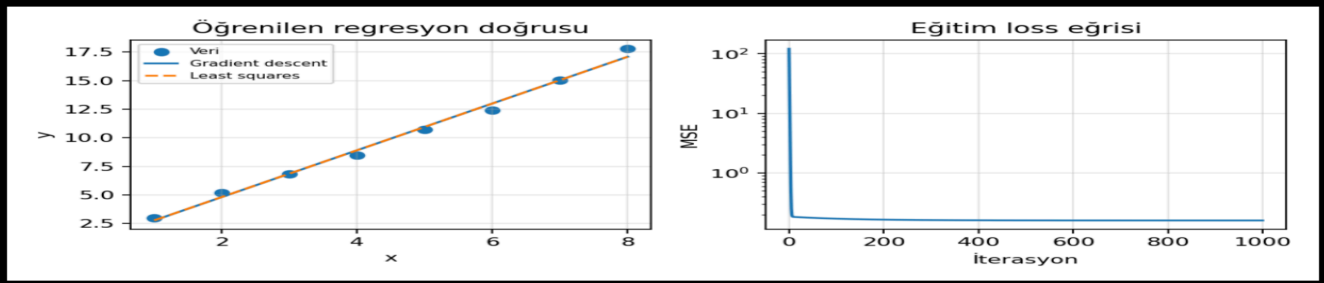
for it in range(1000):
    y_hat = b0 + b1 * x
    error = y_hat - y
    loss = np.mean(error ** 2)

    # ■ki gradient de ayn■ eski parametrelerle hesaplan■r
    db0 = 2 * np.mean(error)
    db1 = 2 * np.mean(error * x)
    loss_history.append(loss)

    # Parametreler birlikte güncellenir
    b0 = b0 - lr * db0
    b1 = b1 - lr * db1

print(b0, b1, loss_history[-1])
plt.semilogy(loss_history)
plt.xlabel("■terasyon")
plt.ylabel("MSE")
plt.grid(True)
plt.show()
```

Ders dokümanındaki çıktılar: $\beta_0 \approx 0,720045$; $\beta_1 \approx 2,047158$; loss≈0,159712. Başlangıç loss'u yaklaşık 120,8177'dir. İlk birkaç iterasyonda loss hızla düşer; sonrasında parametreler daha ince ayar yapar.



Şekil: Solda veri ve öğrenilen doğrular; sağda logaritmik ölçekte loss eğrisi. Eğitim eğrisi, optimizasyonun gerçekten ilerleyip ilerlemediğini anlamak için temel tanı aracıdır.

12. Feature scaling ve optimizasyon

Feature scaling, giriş değişkenlerinin ölçeklerini birbirine yaklaştırır. Örneğin ev fiyatı probleminde alan 50–200 m², oda sayısı 1–5 aralığında olabilir. Bu iki feature arasında yaklaşık 50 kat ölçek farkı vardır.

Ölçekler çok farklıysa loss yüzeyinde bir yönde çok dik, diğer yönde daha yatık bir vadi oluşabilir. Gradient descent dik yönde zikzak yaparken yatık yönde yavaş ilerler. Standardizasyon, vadinin geometrisini iyileştirerek daha dengeli adımlar sağlayabilir.

Z-skor standardizasyonu: x_std=(x−mean(x))/std(x). Dönüşümden sonra ortalama yaklaşık 0, standart sapma yaklaşık 1 olur.

Önemli yorum: Standardizasyon parametrelerin anlamını değiştirir. Ham x ile öğrenilen β_1, bir birim ham x artışının etkisidir; standardize x ile bulunan katsayı, bir standart sapmalık değişimin etkisidir. Bu nedenle katsayıları doğrudan kıyaslamak yerine tahminleri ($\hat{y}$) ve loss'u karşılaştırın.

Ders dokümanındaki örnekte ham veriyle condition number 334.688 iken standardize veride 1'e düşer; yakınsama davranışı belirgin biçimde iyileşir. Bu, ölçeklemenin yalnızca PCA için değil, optimizasyon için de önemli olduğunu gösterir.

13. Gradient checking

Gradient'i elle yazarken işaret veya katsayı hatası yapılabilir. Sonlu fark yaklaşımı, analitik türevi küçük bir ε ile sayısal olarak yaklaşık hesaplayıp karşılaştırır:

$f'(x) \approx [f(x+\varepsilon)-f(x-\varepsilon)]/(2\varepsilon)$, burada ε örneğin 10^{-5} seçilebilir.

```
def f(v):
    return v[0]**2 + 3 * v[1]**2

v = np.array([1.5, -0.8])
analytic = np.array([2*v[0], 6*v[1]])

eps = 1e-5
numeric = np.zeros(2)
for i in range(2):
    e = np.zeros(2)
    e[i] = eps
    numeric[i] = (f(v + e) - f(v - e)) / (2*eps)

print(analytic)  # [ 3.  -4.8]
print(numeric)   # yakla■■k [ 3.  -4.8]
print(np.abs(analytic - numeric).max())
```

Bu kontrol eğitim döngüsünün içinde kullanılmaz; yalnızca kendi gradient fonksiyonunu test etmek için kullanılır. Sayısal yaklaşım yavaştır ama türev kodundaki hataları yakalamaya yarar.

14. İleri optimizer'lara kısa bakış

Momentum: Yalnızca o anki gradient'e bakmak yerine geçmiş hareket yönünü de hesaba katar. Yuvarlanan top benzetmesiyle, hareketin sürekliliğini kullanarak dar vadilerde zikzakları azaltabilir.

Adam: Momentum benzeri geçmiş gradient bilgisini ve parametre başına uyarlanan adım büyüklüklerini birleştirir. Bu ders kapsamında denklemleri işlenmez. Adam birçok problemde iyi bir varsayılan olabilir ancak her zaman en iyi olduğu söylenemez.

15. Model, loss, optimizer ve metric

Kavram	Cevapladığı soru	Örnek
Model	Tahmini nasıl üretiyorum?	Linear regression, sinir ağı
Loss	Ne kadar yanlışım?	MSE, MAE
Optimizer	Parametreleri nasıl değiştireyim?	Gradient descent, SGD, Adam
Metric	Sonuç ne kadar iyi?	RMSE, R^2

Bu dört rol birbirinden bağımsız seçilebilir. Aynı model farklı loss ile eğitilebilir; aynı loss farklı optimizer ile minimize edilebilir. Metric, eğitimde kullanılan loss'tan farklı da olabilir. “Modelim gradient descent” ifadesi yanlıştır; gradient descent model değil optimizer'dır.

16. Kapalı form least squares mi, gradient descent mi?

Kapalı form çözüm, doğrusal regresyonda parametreleri doğrudan hesaplar. Gradient descent ise başlangıç noktasından başlayıp adım adım yaklaşır.

Özellik	Kapalı form / least squares	Gradient descent
Çalışma biçimi	Doğrudan çözüm	İteratif güncelleme
Kapsam	Belirli doğrusal/kare hata problemleri	Çok geniş; doğrusal olmayan modeller de
Ölçekleme	Bu optimizasyon adımlarındaki gibi hassasiyet gösterir	Yakınsamayı güçlü etkiler
Ayar	Learning rate gerekmez	η, iterasyon, batch size gerekir
Büyük modeller	Matris işlemleri pahalı olabilir	Gradient hesaplayarak ölçeklenebilir

Ders verisi için 1000 iterasyonluk gradient descent yaklaşık β_0=0,720045, β_1=2,047158, MSE=0,159712 verir. np.linalg.lstsq ve scikit-learn LinearRegression yaklaşık β_0=0,726429, β_1=2,046071 sonucunu verir. Gradient descent'in farkı, 1000 iterasyonda tam optimuma ulaşmamış olmasından kaynaklanır.

```
X = np.column_stack([np.ones(len(x)), x])
beta_ls, *_ = np.linalg.lstsq(X, y, rcond=None)
print(beta_ls)  # yakla■■k [0.726429, 2.046071]
```

Bu yöntemler rakip değildir. Küçük doğrusal regresyonda least squares doğrudan ve kesin bir çözüm sunabilir. Çok büyük parametreli sinir ağlarında normal equation için gereken dev matris işlemleri pratik olmayabilir; gradient tabanlı optimizasyon bu nedenle temel yöntemdir.

17. Sık yapılan hatalar ve teşhis

Belirti	Muhtemel sebep	Kontrol / çözüm
Loss sürekli artıyor	η büyük veya işaret yanlış	η'yi küçült; güncellemede eksi işaretini denetle
Loss dalgalanıyor	η sınırda veya SGD gürültüsü	η'yi azalt; batch size'ı incele
Çok yavaş öğreniyor	η küçük veya feature ölçekleri farklı	η taraması ve standardizasyon

Belirti	Muhtemel sebep	Kontrol / çözüm
inf / nan	Iraksama veya sayısal taşma	İlk iterasyonlardan loss'u yazdır; finite kontrolü ekle
Gradient beklenmedik	Türev/indeks/ortalama hatası	Gradient checking yap
β değerleri tuhaf	Feature dönüşümü veya ölçek farkı	Parametre yerine tahminleri karşılaştır

Pratik kural: Training loss eğrisini her zaman kaydedin. Yalnızca son loss değerine bakmak, modelin nasıl öğrendiğini ve hangi aşamada sorun çıktığını göstermez.